

Managing Files and Directories:

File Types

RHEL supports several different types of files. Some of the common file types are—regular files, directory files, executable files, symbolic link files, and device files—and are described in the following sub-sections.

1. Regular Files

Regular files may contain text or binary data. These files may be shell scripts or commands in the binary form. When you list a directory, all line entries for files in the output that begin with the `-` sign represent regular files:

```
# ll /bin
-----
-rwxr-xr-x. 1 root root 15392 Feb 12 2014 lastlog
-rwxr-xr-x. 1 root root 19912 Jan 26 2014 lchfn
-rwxr-xr-x. 1 root root 15776 Jan 26 2014 lchsh
```

You can use a command called *file* to determine the type of a file. For example, the following shows that *.bash_profile* in the *root* user's home directory contains *ascii* text:

```
# file .bash_profile
.bash_profile: ASCII text
```

2. Directory Files

Directories are logical containers that are used to hold files and sub-directories. Run *ll* on the */root* directory and you should see an output similar to:

```
# ll /root
-----
drwxr-xr-x. 2 root root 6 Oct 29 08:00 Desktop
drwxr-xr-x. 2 root root 6 Oct 29 08:00 Documents
drwxr-xr-x. 2 root root 6 Oct 29 08:00 Downloads
```

The letter *d* at the beginning of each line entry identifies the file as a directory. You can use the *file* command to see what it reports it as. For example, the following shows that */root* is a directory file:

```
# file /root
/root: directory
```

3. Executable Files

Executable files could be commands in the binary format or shell scripts. In other words, any file that can be run is an executable file. A file that has an *x* in the fourth, seventh, or the tenth field in the output of the *ll* command is executable.

```
# ll /usr/sbin
-----
-rwxr-xr-x. 1 root root 46456 Mar 17 2014 whatis
-rwxr-xr-x. 1 root root 20656 Mar 28 2014 whereis
-rwxr-xr-x. 1 root root 24336 Jan 27 2014 which
```

The *file* command reports the type of an executable file as follows:

```
# file /usr/bin/whoami
/usr/bin/whoami: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked (uses shared libs), for GNU/Linux 2.6.32,
BuildID[sha1]=0x130a11a3f5598dddb581134373700d6554fa6527, stripped
```

4. Symbolic Link Files

A *symbolic link* (a.k.a. a *soft link* or a *symlink*) may be considered a shortcut to another file or directory. When

you issue *ll* on a symbolically linked file or directory, you will notice two things. One, the line entry begins with the letter *l*; and two, there is an arrow pointing to the linked file or directory. For example:

```
# ll /usr/sbin/vigr
lrwxrwxrwx. 1 root root 4 Oct 28 22:47 /usr/sbin/vigr -> vipw
```

The *file* command can be used to confirm that the *vigr* file is a symlink:

```
# file /usr/sbin/vigr
/usr/sbin/vigr: symbolic link to `vipw'
```

5. Device Files

Each piece of hardware in the system has an associated file used by the kernel to communicate with it. This type of file is called a *device file*. There are two types of device files: a *character* (or *raw*) device file and a *block* device file. The following example uses the *ll* command to display device files:

```
# ll /dev/sd*
brw-rw----. 1 root disk 8, 0 Nov 17 14:22 /dev/sda
brw-rw----. 1 root disk 8, 1 Nov 17 14:22 /dev/sda1
brw-rw----. 1 root disk 8, 2 Nov 17 14:22 /dev/sda2
# ll /dev/usb*
crw-----. 1 root root 251, 0 Nov 17 14:22 /dev/usbmon0
crw-----. 1 root root 251, 1 Nov 17 14:22 /dev/usbmon1
crw-----. 1 root root 251, 2 Nov 17 14:22 /dev/usbmon2
```

The first character in each line entry tells if the file type is block or character. A *b* denotes a block device file and a *c* represents a character device file. You can use the *file* command to determine their types:

```
# file /dev/sda
/dev/sda: block special
```

```
# file /dev/tty0
/dev/usbmon0: character special
```

File and Directory Operations

This section elaborates on various management operations that can be performed on files and directories. These operations include creating, listing, displaying contents of, copying, moving, renaming, and deleting files and directories. These are common operations that *root* and all normal users usually perform.

Creating Files and Directories

Files can be created in multiple ways; however, there is only one command to create directories.

Creating Files Using the touch Command

The *touch* command creates an empty file. If the file already exists, it updates the time stamp on it to the current system date and time. Execute the following as *user1* in their home directory to create *file1*, and then run the *ll* command to verify:

```
$ cd
$ touch file1
$ ll file1
-rw-rw-r--. 1 user1 user1 0 Nov 26 10:07 file1
```

As expected, the fifth field in the output is 0, meaning that *file1* is created with zero bytes. Now, if you re-run the command on *file1*, you will notice that it gets an updated time:

```
$ touch file1
```

```
$ ll file1
```

```
-rw-rw-r--. 1 user1 user1 0 Nov 26 10:14 file1
```

Creating Files Using the cat Command

The *cat* command allows you to create short text files:

```
$ cat > newfile
```

Nothing is displayed when you execute this command because the system is waiting on you to input something; it expects you to enter text that it will capture into *newfile*. Press Ctrl+d when done to save and return to the shell prompt.

Creating Files Using the vi (vim) Editor

```
$ vim file10
```

```
$ vi file19
```

Creating Directories Using the mkdir Command

The *mkdir* command is used to create directories. This command shows an output if you run it with the *-v* option. The following example demonstrates the creation of a directory called *scripts.dir1* in *user1* user's home directory (*/home/user1*):

```
$ cd
```

```
$ pwd
```

```
/home/user1
```

```
$ mkdir scripts.dir1
```

```
mkdir: created directory 'scripts.dir1'
```

You can create a hierarchy of sub-directories by specifying the *-p* option with *mkdir*. In the following example, *mkdir* creates the hierarchy *scripts.dir2/perl/per5* in *user1*'s home:

```
$ mkdir -p scripts.dir2/perl/per5 -v
mkdir: created directory 'scripts.dir2'
mkdir: created directory 'scripts.dir2/perl'
mkdir: created directory 'scripts.dir2/perl/per5'
```

Listing Files and Directories

Use the *ll* command to list files and directories. This command shows the details for each listed file and directory in nine columns. For instance:

Displaying File Contents

Linux offers several tools for showing file contents. Directory contents are simply the files and sub-directories that it contains. Use the *ll* or the *ls* command as explained earlier to view directory contents.

For file viewing, you can use the *cat*, *more*, *less*, *head*, and *tail* commands. These tools are explained below.

Using the cat Command

cat displays the contents of a text file. In the example below, the *.bash_profile* file in *user1*'s home directory is displayed with the *cat* command:

```
$ cat /home/user1/.bash_profile
```

```
# Get the aliases and functions
```

```
if [ -f ~/.bashrc ]; then
```

```
. ~/.bashrc
```

```
fi#
User specific environment and startup programs
PATH=$PATH:$HOME/.local/bin:$HOME/bin
export PATH
```

Using the more and less Commands

more can be used to view long text files one page at a time, starting at the beginning. In the example below, */etc/profile* is displayed with *more*. This command shows the percentage of the file being viewed in the last line.

\$ more /etc/profile

```
.....
USER=""id -un""
--More--(43%)
```

Navigation keys listed in Table are helpful while you are in a *more* session.

Key	Purpose
Spacebar / f	Scrolls forward one screen.
Enter	Scrolls forward one line.
b	Scrolls backward one screen.
d	Scrolls forward half a screen.
h	Displays help.
q	Quits and returns to the command prompt.
/string	Searches forward for a string.
?string	Searches backward for a string.
n / N	Finds the next / previous occurrence of a string.

less is similar to the *more* command but it offers some extended capabilities. It does not need to read the entire file before it starts to show it, thus making it faster than *more*. In the example below, */etc/profile* is shown with *less*:

\$ less /etc/profile

```
.....
USER=""id -un""
/etc/profile
```

The same navigation keys listed in Table 3-1 are also helpful when viewing a large file with *less*.

Using the head and tail Commands

head displays the first few lines of a text file. By default, it shows the first ten lines. See the example below:

\$ head /etc/profile

```
# /etc/profile
# System wide environment and startup programs, for login setup
# Functions and aliases go in /etc/bashrc
# It's NOT a good idea to change this file unless you know what you
# are doing. It's much better to create a custom.sh shell script in
# /etc/profile.d/ to make custom changes to your environment, as this
# will prevent the need for merging in future updates.
```

There are three empty lines in the above output as well. You can specify a number with the command as an argument and it will show you that many lines only. For example, to view the first three lines from */etc/profile*:

\$ head -3 /etc/profile

```
# /etc/profile
# System wide environment and startup programs, for login setup
```

tail, on the other hand, displays the last ten lines of a file by default unless a number is supplied to alter its behavior. The following example shows the last ten lines from */etc/profile*:

And the following example shows only the last three lines from the same file:

\$ tail -3 /etc/profile

```
done
unset i
unset -f pathmunge
```

The *tail* command proves to be very useful when you wish to view a log file while it is being updated. The *-f* option enables this function. The following example shows how to view the system log file */var/log/messages* in real time:

```
# tail -f /var/log/messages
```

Copying Files and Directories

The copy operation duplicates a file or directory. There is a single command called *cp* that is used for this purpose.

Copying Files

The *cp* command copies one or more files to either the current directory or another. If you want to duplicate a file in the same directory, you must give a different name to the target file. If you want to copy a file to a different directory, you can use the same file name or different. Consider the following examples.

To copy *file1* as *newfile1* in the same directory:

```
$ cp file1 newfile1
```

To copy *file1* by the same name to another existing directory called *scripts.dir1*:

```
$ cp file1 scripts.dir1
```

By default, the copy operation overwrites the destination (if it exists) without giving a warning. Use the *-i* option to instruct the *cp* command to prompt for confirmation before overwriting:

```
$ cp -i file1 scripts.dir1
```

cp: overwrite `scripts.dir1/file1'?

By default, you do not need to specify the *-i* option for yes/no confirmation if you are attempting to copy a file to overwrite the destination file as *root*.

There is a pre-defined alias “`alias cp='cp -i'`” in the *root* user’s *~/.bashrc* file that takes care of that.

Copying Directories

The *cp* command with the *-r* (recursive) option copies an entire directory tree to another location. In the following example, *scripts.dir1* is copied to *scripts.dir2* and then the directory contents of *scripts.dir2* is displayed for validation:

```
$ cp -r scripts.dir1 scripts.dir2
```

```
$ ll scripts.dir2
```

drwxrwxr-x. 2 user1 user1 18 Nov 26 11:27 scripts.dir1

You may want to use the *-i* option for overwrite confirmation.

Moving and Renaming Files and Directories

A file or directory can be moved within the same file system or to another. Within the file system move, a corresponding entry is added to the target directory and the source entry is removed. In this case, the actual data remains intact. On the other hand, a move to a different file system physically moves the file or directory contents to the new location.

A rename simply changes the name of a file or directory; data is not touched.

Moving and Renaming Files

The *mv* command is used to move or rename files. The *-i* option can be specified for user confirmation if a file by that name already exists. The following example moves *file1* to *scripts.dir1* and prompts for confirmation:

```
$ mv -i file1 scripts.dir1
```

mv overwrite: scripts.dir1/file1? (y/n)

By default, you do not need to specify the *-i* option for yes/no confirmation if you are attempting to move a file to overwrite the destination file as *root*.

There is a pre-defined alias “`alias mv='mv -i'`” in the *root* user’s *~/.bashrc* file that takes care of that.

To rename *newfile* as *newfile1*:

```
$ mv newfile newfile1
```

You can use the *-i* option with *mv*, if needed.

Moving and Renaming Directories

To move a directory along with its contents to somewhere else or simply change the name of the directory, use the *mv* command. For example, to move *scripts.dir1* into *scripts.dir2* (*scripts.dir2* must exist, otherwise it will simply rename it), issue the following:

```
$ mv scripts.dir1 scripts.dir2
```

To rename *scripts.dir2* as *scripts.dir20*:

```
$ mv scripts.dir2 scripts.dir20
```

Removing Files and Directories

The remove operation deletes a file entry from the directory structure and marks its data space as free. For a directory, it deletes corresponding entries from the file system structure.

Removing Files

You can remove a file using the *rm* command, which deletes one or more specified files at once. The following example deletes *newfile*:

```
$ rm newfile
```

The *-i* option can be used to prevent accidental removals. The option instructs the command to prompt for confirmation before proceeding with the removal. See the following example:

```
$ rm -i newfile
```

rm: remove regular empty file 'newfile'?

By default, you do not need to specify the *-i* option for yes/no confirmation if you are attempting to remove a file as *root*. There is a pre-defined alias “alias *rm='rm -i'*” in the *root* user's *~/.bashrc* file that takes care of that.

Removing Directories

There are two commands available to remove directories, which are demonstrated in the following examples.

Use the *rmdir* command to remove an empty directory:

```
$ rmdir scripts.dir20
```

Use the *rm* command with the *-r* switch to remove a non-empty directory:

```
$ rm -r scripts.dir10
```

You can provide the *-i* option with *rm* to instruct the command to prompt for confirmation for each file being deleted.

Muhammad Adel